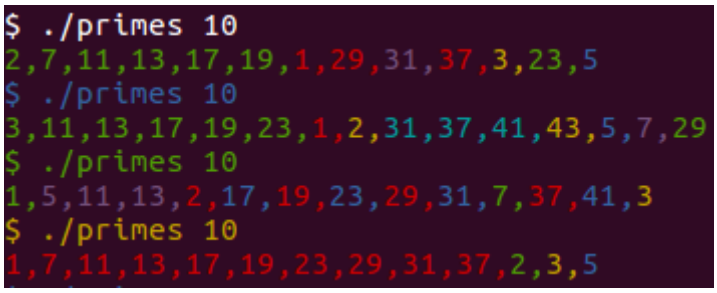
# OS Lab exercises on using Condition Variable and Semaphore

**Q1.** Implement the Producer/Consumer problem using condition variable for managing a buffer represented by a stack. Assume the following:

- The buffer is a stack with at most 5 elements. Each element has a random integer value (between 0 to 30). Initially the stack will be empty.
- At the start of the program, the producer will push 5 (maximum allowed number of elements) elements to the stack and print a message with its value to the screen. Once the stack is full, producer should print a message and wait.
- Consumer will perform pop operation on the stack and print a message with its value to the screen. When the stack is empty, it should print a message and wait.

**Q2. Parallel Prime Number Generation**: You are given a starter code for concurrent prime number generation implementation. The program generates prime numbers with up to 8 pthreads (you can change it to set to the number of cores your system has). Inspect the code carefully. Try to understand how it operates. If you compile and run it, you will note that each thread prints the prime numbers it generates using a different color. However, out of the box we notice there are some issues. For example, If you specify "10" as a command line argument to restrict output to 10 prime numbers, we seem to get a different number of prime numbers each time! (see figure)




Your first task is to correct this problem by introducing a state variable to track when any of the threads creates the $n^{th}$ prime number. This is the number of primes the user has requested to produce. Also, introduce a mutex_lock and condition variable to signal the parent thread when the $n^{th}$ prime number has been generated. The main program should then terminate the program (you can comment out/remove calls to pthread_join () in that case). Also, record the time to generate 50000 prime numbers ($ **`time ./primes 50000)`** with output enabled and disabled (by changing the OUTPUT compiler constant)

Next, you would have observed that the sequence of prime numbers printed is not in correct order. This is because random CPU context switches occur at any time causing threads to be interrupted between when they initially grab a number to check if it's prime, and when prime numbers are finally reported by printing their value to the screen. So, your second task is to enforce a strict ordering of prime number generation using a lock variable to force: (1) finding a prime number, and (2) reporting it to the screen, to be a **single atomic operation.** Time how long it now takes to generate 50,000 prime numbers. You will realize a trade-off in this program between **performance** and **correctness**. If we don't care for the ordering, the benefit is that the code can scream through prime generation using

this non-optimal algorithm fairly quickly if we only care about generating any primes. Without changing the prime generation algorithm, contemplate ways to improve the performance of the program, if any.

**Q3. The Readers-Writers Problem.** Consider an application where multiple threads of a process wish to read (reader threads) and write (writer threads) some shared data. There can be two ways to design such systems, with *reader preference* (when a reader thread is accessing the data, additional readers are allowed to concurrently hold the lock with previous readers even if a writer thread is waiting for the lock) or *writer preference* (additional readers are not granted the lock when a writer is already waiting). Write a program to simulate above mentioned application with ***writer preference (****as reader preference code* has been discussed in the class). Use PTHREAD library and semaphores to implement the synchronization. You can refer to below table to understand the rules of *writer preference*.

| **Thread accessing shared data** | **New request** | **Preferred action** |
|---|---|---|
| None | Reader | Starts reading. |
| None | Writer | Starts writing. |
| Only reader/readers | Writer | Writer blocked until readers **currently accessing** the database are finished. |
| Only reader/readers | Reader | Starts reading if no writers are blocked. Otherwise reader will be blocked until all writers complete writing. |
| Writer | Writer | New writer is blocked. Only one writer may write the data at a time. |
| Writer | Reader | Reader blocked. |
| Writer accessing, but readers blocked | Writer | Writer is blocked, but should write before any blocked readers (no matter when they arrive) are allowed to read. |

You can use the given skeleton code given in (readers-writers_writer-pref.c) and test your code on request mentioned in below table.

| **Thread** | R1 | R2 | R3 | W1 | R4 | R5 | W2 | R6 | R7 |
|---|---|---|---|---|---|---|---|---|---|
| **Arrival time** | 0 | 2 | 4 | 5 | 6 | 7 | 7 | 8 | 9 |
| **Execution time** | 8 | 6 | 5 | 6 | 2 | 2 | 4 | 2 | 3 |